Retinal Screening and Referral Support: Final Product Technical Report

Michael Kimani · BSc Software Engineering (ML), African Leadership University · Supervisor: Emmanuel Adjei · 26 October 2026
Repository: github.com/Mikekimm/michael-kimani-eye-screening-capstone (branch `main`) · Video: **[link]**

Scope of claims. This is screening and referral *support*, not a diagnostic device. The model was trained and evaluated only on public datasets; there has been no field deployment or clinical validation. Every figure below comes from a recorded run in the repository.

1. Scope alignment and feature validation

The approved proposal planned an offline-first Android app that screens fundus photos for diabetic retinopathy (DR), glaucoma and cataract on the device, refers uncertain or high-risk cases to a specialist dashboard, and syncs when connectivity returns. Following supervisor feedback (a feasible prototype with one demonstrable workflow; reconcile offline use with sync and authentication; separate research evaluation from development), effort went into one complete, tested DR workflow, while keeping the multi-disease design real: glaucoma and cataract are registered conditions that report "Model not available yet" until a trained model is added, which needs no code change. The full table with evidence paths is in `docs/scope_table.md`.

Planned feature	Intended purpose	Status	Evidence
DR classifier (MobileNetV2, TFLite)	Detect DR from a fundus photo	Completed	`models/dr/`, real-model API tests
Glaucoma and cataract classifiers	Multi-disease screening	Partial	Patient-separated ODIR-5K training notebook ready and smoke-tested; run pending. App shows "not available" until a model is added (no code change)
Five-band cataract grading	Grade cataract severity	Not implemented	ODIR-5K has no severity labels; glaucoma and cataract are binary only rather than showing invented bands
Risk / severity output	Show urgency	Changed	DR: five-band *estimated* band from the binary score (`RiskPolicy`, boundary tests); others binary
Sensitivity ≥ 85%, specificity ≥ 80%	Safe screening trade-off (RQ3)	Not met	Validation fold: sensitivity 0.542, specificity 0.97 at threshold 0.5; hold-out and calibration not completed
On-device inference	Screen without internet	Changed	Server-side LiteRT; offline photos queued and analysed after sync
Android app (React Native)	Mobile client	Changed	Installable PWA; E2E on Android and iPhone emulation
Register patient, capture image, view result	Core health-worker workflow	Completed	E2E journey 1, screenshots
Offline storage and two-way sync	Work with intermittent connectivity	Completed	IndexedDB queue, idempotent push, decision pull; 10 sync tests
Flag and refer uncertain / high-risk cases	Structured referral	Completed	Automatic and manual referral, workflow tests
Specialist review: approve or override	Close the referral loop	Completed	Plus retake / comment; append-only history; E2E journey 2
Admin: users and analytics	Access control, patterns	Completed	Role management, aggregate-only analytics
FastAPI + PostgreSQL backend	Central store and API	Partial	FastAPI + SQLite done; PostgreSQL via `DATABASE_URL` and Docker written, not yet run
Load balancer	Scale to many clinics	Not implemented	Stateless JWT API can scale horizontally
Sync tested under interruptions	Reliability (objective 4)	Completed	Lost-reply, offline-queue and backoff tests
Usability test with health workers	Check usability	Not yet	SUS study planned

Added beyond the proposal: authentication with role-based access (Argon2, JWT refresh rotation, login throttling), an image quality gate, a SHA-256-verified model registry with model cards, eye laterality and model version on every result, an append-only review log and an audit log (the last three answer the supervisor's data-model corrections).

2. Technical functionality

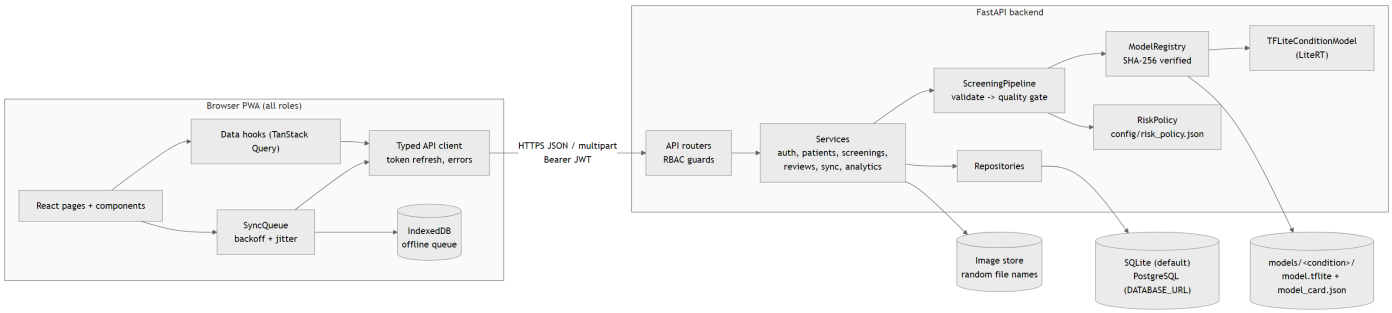


Figure 1. Architecture as implemented (generated from `docs/diagrams/src/architecture.mmd`).

Frontend. A React 18 + TypeScript (strict) PWA, mobile-first (bottom tabs below 1024 px, sidebar above). Pages read server data only through hooks that call one typed API client, which attaches the JWT, refreshes it once on a 401 for all concurrent requests, and converts failures into plain-language messages. Health workers register patients by clinic code, choose the eye, and capture a photo with a circular guide (rear camera or gallery); on-device hints flag dark or blurry photos before upload.

Backend. FastAPI with a layered design: routers (HTTP and role guards) → services (use cases) → repositories (all SQL). `POST /api/screenings` checks type, size and that the bytes really decode as JPEG/PNG, applies the quality gate (blank or tiny images return 422 with a reason), then runs every model the registry has verified and maps each score through that condition's risk policy. One `diagnosis_results` row is stored per condition with its model version, including "not available" rows, so history shows exactly what ran. Positive or borderline results open a referral; specialists approve, override (reason required), request a retake or comment, and each action is appended to `review_events`, which the ORM refuses to update or delete.

Offline and sync. Without a connection, photos and new patients are stored in IndexedDB with a client-generated UUID. A sync engine pushes them in order (patients first), retrying with bounded exponential backoff and full jitter; because the server is idempotent on the UUID, a retry after a lost reply returns the original record rather than a duplicate. Each cycle then pulls specialist decisions made since the last pull. Offline mode captures and queues; it does not run the model on the device.

Data handling and privacy. SQLite by default, PostgreSQL by configuration, schema managed by Alembic migrations. Patients are pseudonymous codes; images are stored under random names and served only to authorised users; analytics and audit records contain no images or patient fields; secrets live only in a git-ignored `.env` generated by a script.

3. Code quality, OOP and algorithms

Principle / pattern	Where
Abstraction + polymorphism	`ConditionModel` → `TFLiteConditionModel`; tests substitute a labelled double
Strategy	`RiskPolicy` per condition, built by `RiskPolicyFactory` from JSON
Open/closed	`ModelRegistry`: new condition = new folder
Repository + DI	backend repositories; `QueueRepository` (IndexedDB / in-memory); `api/deps.py`
Encapsulation	append-only history enforced by ORM listeners; one `AccessPolicy` for record visibility
Single responsibility	routers, services, repositories, presenters, schemas in separate modules

Algorithm	Purpose and complexity
Laplacian variance, brightness, vignette test	quality gate, O(pixels) on a 256^2 copy
Sensitivity-targeted threshold search	highest threshold meeting 85% sensitivity, O(n log n) via sort + binary search
Rank-sum AUC, stratified bootstrap CIs	honest evaluation, NumPy only; AUC matches scikit-learn
Threshold-anchored severity bands	bands can never contradict the decision (1,001-score sweep test)
Idempotent sync, backoff with full jitter	no duplicates, no retry storms
Streaming SHA-256	model integrity, 1 MiB chunks

Quality practices: type hints throughout the backend and TypeScript strict mode with ESLint (which forbids `fetch` outside the API client); Pydantic v2 schemas separate from ORM entities; centralised configuration (settings, risk policy, model cards); small conventional commits on a feature branch (`git log`); a reproducible environment (pinned requirements, `package-lock.json`). Superseded Streamlit code was removed and tagged in git rather than left alongside.

4. Testing approaches and results

Strategy	Why chosen	How	Result
Unit (algorithms)	boundary errors would mis-triage silently	pytest, Vitest; boundary values, property sweep; deliberate code breakage to check tests fail	94 + 25 pass
Integration (API + DB + real model)	security and integrity rules live between layers	fresh DB per test; real LiteRT model; 45-case role-by-route matrix; 9 invalid-upload cases	90 pass
End-to-end functional	prove users can finish clinical tasks	Playwright: worker journey, specialist review, unauthorized access, invalid input	16/16 pass

Compatibility	clinics use varied phones and browsers	same journeys on Chromium (Pixel 5, desktop), Firefox, WebKit (iPhone 13)	all engines pass
Performance	slow hardware and networks	benchmark script; throttled browser round trip; Lighthouse	see section 5
Accessibility	low-vision and screen-reader users	axe-core WCAG 2.1 A/AA on 15 screens, all roles, mobile and desktop	0 violations
Security (brute force)	password guessing	sliding-window login throttle; unit tests with a fake clock and API tests	429 after 5 failures
Model validation	RQ3, honest accuracy claims	validation fold of the training notebook (hold-out notebook prepared, run not finished)	target not met

Different inputs: scores of 0.05, 0.45 and 0.9 (negative, borderline, positive), blank, tiny, corrupt, mislabelled (BMP sent as JPEG), GIF, oversized and future-dated uploads, name-like patient codes, expired, replayed and wrong-type tokens, and every role against every route. **Improvements made because of testing:** a form test exposed that the password field dropped its form reference, so sign-in could never succeed (fixed with `forwardRef`); an end-to-end script test exposed that the calibration script did not write its outputs (fixed); screenshots led to listing DR first and to plain-language referral reasons; Lighthouse raised SEO from 91 to 100; axe-core found unlabeled camera and gallery inputs (fixed); a relative database path that created two databases was anchored.

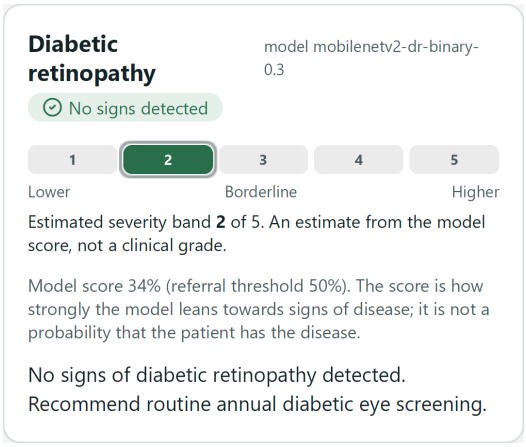


Figure 2. DR result card on iPhone 13 (WebKit), from a synthetic test image: estimated band, model version, score explained, action.

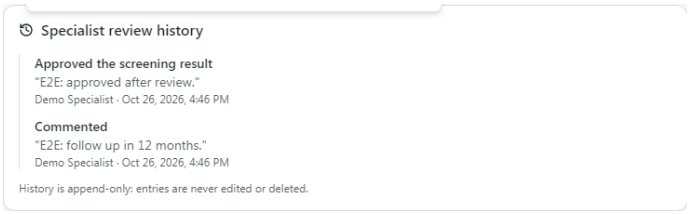


Figure 3. Specialist review history (append-only), desktop Chrome.

5. Performance and compatibility

Measurement (Windows 11, Intel CPU, 8 logical cores)	Result
Model inference, LiteRT, 200 runs (all cores / 1 thread)	55 / 60 ms mean, 68 / 84 ms p95; 2.6 MB model; about 117 MB process memory
Full server pipeline on a 1024 px JPEG (all cores / 1 thread)	74 / 77 ms mean (an earlier run under background load: 166 ms)
Upload to result in the browser (median of 3)	0.50 s; 3.17 s with emulated 6x slower CPU and slow 3G
Lighthouse, sign-in page (performance / accessibility / best practices / SEO)	mobile 96 / 100 / 100 / 100; desktop 100 / 100 / 100 / 100
Accessibility of signed-in screens (axe-core, WCAG 2.1 AA)	0 violations on 15 screens, mobile and desktop
Browsers and devices (Playwright)	Chromium on Pixel 5 and desktop, Firefox desktop, WebKit on iPhone 13: all journeys pass
Bundle	route-level code splitting; core about 110 KB gzipped; charts (103 KB) load only for admins

Analysis: inference is a small share of the round trip, and limiting it to one thread costs about 5 ms, so a modest server CPU is sufficient. On a slow phone and network the wait rises to about 3 s, dominated by the upload; the offline queue means the worker can continue with the next patient meanwhile. Low-end figures are emulated (Chrome DevTools); physical low-end devices and real iOS Safari have not been tested.

6. Model evaluation, limitations and next steps

The DR model is MobileNetV2 (frozen ImageNet backbone, small head) trained for 8 epochs on the Kaggle EyePACS + APTOS + Messidor redistribution with a group-aware 5-fold split. The training notebook reported, on its 47,234-image validation fold at threshold 0.5: accuracy 0.790, precision 0.920, recall 0.542, F1 0.682. These are **validation results from the training notebook, not an independent hold-out**: the same fold guided early stopping, and the independent hold-out run did not finish. Specificity from the same report is 0.97. The **85% sensitivity target was not met** at threshold 0.5: about 46% of DR images in

that fold were missed, which is unacceptable for screening on its own. The prepared remedy is to calibrate the threshold for 85% sensitivity on one half of a verified hold-out and report on the other half, accepting more referrals; this was not completed. The app, model card, About screen and README show these figures with the same label.

- Evaluation is image-level (no patient IDs), the redistribution contains augmented near-duplicates, and the validation fold also guided early stopping, so results are optimistic; there is no external test set and no smartphone-lens images.
- Glaucoma and cataract models depend on the ODIR-5K notebook run (patient-separated split, threshold from validation, test scored once); their results will be reported as measured, even if weak. Offline inference in the browser, PostgreSQL/Docker runs, a usability study with health workers and tests on physical low-end devices remain to be done.
- The TFLite export is quantized and differs from the Keras model by up to 0.034 in score on test inputs; the full-fold difference will be recorded by the hold-out run.

7. Use of AI assistance

This project was designed and directed by Michael Maina Kimani. I wrote the proposal, chose the scope and the honesty rules (real trained models only, no invented results), trained and evaluated the diabetic retinopathy model on Kaggle, and made the decisions on what to build and what to cut.

I used Claude (Anthropic), through Claude Code, to help write, refactor and test the backend and frontend code, the test suites and the documentation, working under my direction. I reviewed the results and I am responsible for the submitted work.